

Trabalho Prático

Sistema de Monitoramento de Falhas em Redes Elétricas

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

O objetivo do trabalho foi desenvolver um sistema distribuído para monitoramento de rede elétrica por sensores, utilizando uma arquitetura baseada em dois servidores distintos — SL (Servidor de Localização) e SS (Servidor de Status) — e múltiplos clientes sensores. A comunicação entre os elementos do sistema ocorre via TCP sobre IPv4, utilizando mensagens com códigos específicos definidos em protocolo próprio.

O sistema foi projetado para simular cenários de monitoramento de falhas elétricas, com sensores instalados em diferentes localizações que enviam atualizações sobre seu estado de risco. A separação entre localização (SL) e risco (SS) permite uma divisão clara de responsabilidades, isolamento de dados e melhor organização da lógica do sistema.

Além da comunicação básica cliente-servidor, o trabalho também implementa comunicação peer-to-peer entre os servidores.

Ao longo da implementação, diversas funcionalidades adicionais foram desenvolvidas para complementar o enunciado original, como: comandos de relatórios, envio de alertas e estado seguro, e listagem de sensores.

2. Mensagens e Protocolo

O sistema de monitoramento de falhas opera sobre o protocolo TCP, garantindo a entrega ordenada e confiável das mensagens. As mensagens de aplicação são curtas, com um tamanho máximo de 500 bytes, e transportam textos codificados em ASCII, sem caracteres acentuados ou especiais.

2.1. Estrutura da Mensagem

Todas as mensagens são estruturadas com um código e um payload.

As funções `send_message` e `recv_message` são responsáveis por enviar e receber mensagens via socket.

2.2. Códigos e Tipos de Mensagens

O protocolo define uma variedade de códigos de mensagem, agrupados por funcionalidade:

Mensagens de Controle:

Utilizadas para gerenciar as conexões entre servidores (peers) e entre clientes (sensores) e servidores.

REQ_CONNPEER (20): Requisição de conexão entre servidores.

RES_CONNPEER (21): Resposta de conexão entre servidores, com o ID do peer (PidS).

REQ_DISCPEER (22): Requisição de desconexão entre servidores, com o ID do peer (PidS).

REQ_CONNSEN (23): Requisição de conexão de um sensor na rede, com sua localização (LocID).

RES_CONNSEN (24): Resposta de conexão de um sensor, com o ID atribuído (IdSen).

REQ_DISCSEN (25): Requisição de desconexão de um sensor na rede, com o ID do sensor (IdSen).

Mensagens de Dados:

Utilizadas para a troca de informações específicas sobre status, localização e alertas de sensores.

REQ_CHECKALERT (36): SS solicita a SL a localização de um sensor em alerta, com SensID.

RES_CHECKALERT (37): SL responde SS com a LocID do sensor em alerta.

REQ_SENSLOC (38): Cliente solicita a SL a localização de um sensor, com SensID.

RES_SENSLOC (39): SL responde cliente com a LocID do sensor.

REQ_SENSSTATUS (40): Cliente solicita a SS o status de um sensor, com SensID.

RES_SENSSTATUS (41): SS responde cliente com o LocID de um sensor em alerta.

REQ_LOCLIST (42): Cliente solicita a SL a lista de sensores em uma LocID.

RES_LOCLIST (43): SL responde cliente com a lista de IDs de sensores (SenIDs) em uma localização.

Mensagens de Dados (extras):

REQ_LISTSTATUS_SERVER (60): Servidor pede a outro servidor a lista de status de sensores.

RES_LISTSTATUS_SERVER (61): Servidor responde a outro servidor com a lista de status (SenIDs Risk).

REQ_LISTLOC_SERVER (62): Servidor pede a outro servidor a lista de localização de sensores.

RES_LISTLOC_SERVER (63): Servidor responde a outro servidor com a lista de localização (SenIDs Loc).

REQ_RISKSSENS_SERVER (64): Servidor pede a outro servidor o risco de um sensor específico (SensID Loc).
RES_RISKSSENS_SERVER (65): Servidor responde a outro servidor com risco (SensID Loc Risk).
REQ_LOCSSENS_SERVER (66): Servidor pede a outro servidor a localização de um sensor específico (SensID Risk).
RES_LOCSSENS_SERVER (67): Servidor responde a outro servidor com localização (SensID Loc Risk).
REQ_UPDRISK (68): Sensor ou servidor envia a SS para atualizar o risco de um sensor (RiskVal).
REQ_LOCUPDATE (69): SS envia a SL para obter todas as localizações de sensores após uma atualização de risco (SensID Risk).
RES_LOCUPDATE (70): SL responde SS com a lista de localizações de todos os sensores (SensID Risk AllLocs).
NOTIFY_DISCSERVER (71): Servidor notifica clientes sobre sua desconexão.
NOTIFY_FAILURE (72): Servidor (SL por multicast) notifica clientes sobre falha/resolução de pane (Message).
REQ_MULTICAST_FAILURE (73): SS solicita a SL multicast de falha para sensores em uma área (Area Message).
REQ_RISKSSENS_SENSOR (74): Cliente solicita a SS o risco de um sensor (SensID).
RES_RISKSSENS_SENSOR (75): SS responde cliente com o risco (RiskVal).
REQ_LISTSTATUS_SENSOR (76): Cliente solicita a SS a lista de status de sensores.
RES_LISTSTATUS_SENSOR (77): SS responde cliente com a lista de status (SenIDs Risk).
REQ_LISTLOC_SENSOR (78): Cliente solicita a SL a lista de localização de sensores.
RES_LISTLOC_SENSOR (79): SL responde cliente com a lista de localização (SenIDs Loc).
REQ_REPORTLOC_SENSOR (80): Cliente solicita a SL a localização para relatório (SensID).
RES_REPORTLOC_SENSOR (81): SL responde cliente com a localização para relatório (LocId).
REQ_REPORTRISK_SENSOR (82): Cliente solicita a SS o risco para relatório (SensID).
RES_REPORTRISK_SENSOR (83): SS responde cliente com o risco para relatório (RiskVal).

Mensagens de Erro ou Confirmação:

Mensagens padrão para sinalizar sucesso ou falha em operações.

ERROR (255): Mensagem de erro. O payload indica o código: **01** ("Peer limit exceeded"), **02** ("Peer not found"), **09** ("Sensor limit exceeded"), **10** ("Sensor not found"), **11** ("Location not found").

OK (0): Mensagem de confirmação. O payload indica o código: **01** ("Successful disconnect"), **02** ("Successful create"), **03** ("Status do sensor 0").

3. Arquitetura do Sistema e Estruturas de Dados

3.1. Componentes do Sistema

- **Sensores (Clientes):** Dispositivos distribuídos na rede elétrica, com a localização (1 a 10) definida por linha de comando pelo usuário ou definida aleatoriamente pelo próprio código. Cada sensor conecta simultaneamente aos dois servidores e são capazes de detectar anomalias e reportar um status binário (1 para risco, 0 para sem risco). Os sensores iniciam sem identificação, recebendo-a dos servidores após a solicitação de entrada na rede, por ser uma conexão simultânea e ocorrer de forma sequencial entre os sensores, cada um recebe o mesmo ID dos dois servidores.
- **Servidor de Localização (SL):** Responsável por armazenar e gerenciar a localização dos sensores (valores inteiros entre 1 e 10). Sensores com localização -1 são considerados fora da zona ativa de cobertura, entretanto pela forma como as localizações são definidas, esse caso nunca ocorre.
- **Servidor de Status (SS):** Responsável por armazenar e gerenciar os status de risco (0 ou 1) dos sensores. Monitora os alertas recebidos dos sensores para detecção de panes e resolução de panes, além de comunicar o SL e os sensores envolvidos quando ocorre.

3.2 Topologia de Comunicação

A comunicação no sistema é estabelecida através de sockets na linguagem C.

Observe o diagrama de comunicação no fim do arquivo, na seção 6. Imagens. Ele resume esta subseção.

- **Comunicação Peer-to-Peer (SL ↔ SS):** Os servidores de localização (SL) e de status (SS) conectam entre si por meio de uma conexão ponto-a-ponto — utilizam a porta 64000, que permite a troca de mensagens para completar informações necessárias aos comandos.
- **Comunicação Cliente-Servidor (Sensor ↔ SL/SS):** Cada sensor (cliente) estabelece e mantém conexões simultâneas com ambos os servidores, SL e SS — SL usa a porta 60000 e SS usa a porta 61000. Isso permite que cada um dos servidores tenha controle independente sobre parte das informações dos sensores (localização ou risco). Cada servidor gerencia seu próprio conjunto de sockets para as conexões com os clientes.

3.3. Estruturas de Dados Comuns

As principais estruturas de dados que facilitam a comunicação e o gerenciamento de informações são:

- **Message:** Representa uma mensagem trocada entre os componentes. Contém os campos:
code: código da mensagem (enum)

payload: conteúdo da mensagem

O tamanho máximo da mensagem é de 500 bytes, sendo 495 para payload e 5 para cabeçalho.

- **SensorInfo:** Representa um sensor conectado ao servidor. Contém:
 - id: identificador único do sensor
 - location: localização (1–10)
 - risk: status de risco (0 ou 1)
 - socket_fd: socket associado ao sensor
- **PendingRequest:** Usada pelo servidor SS para rastrear sensores aguardando resposta do SL. Contém:
 - sensor_id: ID do sensor em risco
 - socket_fd: socket do sensor aguardando resposta
- **ServerContext:** Utilizada no cliente para associar o socket com o tipo de servidor (SL ou SS), facilitando o tratamento das respostas de cada um. Contém:
 - sockfd: descritor do socket
 - label: identificador textual ("SL" ou "SS")

Todos os acessos dados compartilhados entre múltiplas threads são protegidos por mutexes, garantindo a integridade das informações.

3.4. Comandos e Fluxos de Mensagens

O sistema orquestra diversos fluxos de mensagens para gerenciar o monitoramento.

Comandos dos servidores:

- **close connection / kill:** Desconecta o servidor da conexão P2P, mas antes notifica todos os sensores conectados, assim eles também se desconectam, já que a rede só funciona com dois servidores.
Servidor A manda **NOTIFY_DISCSERVER** para todos os sensores, em seguida manda **REQ_DISCPEER** para Servidor B. Sensor recebe **NOTIFY_DISCSERVER**, então manda **REQ_DISCSEN** para Servidor B, que responde **ERROR(10)** se não encontra Sensor, ou **OK(01)** se confirmar desconexão de Sensor. Servidor B recebe **REQ_DISCPEER**, responde **ERROR(02)** se não encontrar Servidor A, ou **OK(01)** se confirmar desconexão do Servidor A.
- **list sensors:** Lista todos os sensores conectados na rede, separados por região, e informa a localização e o status de risco de cada um. Para isso, é preciso solicitar as informações faltantes para o outro servidor.
Se SL: envia **REQ_LISTSTATUS_SERVER** para SS pedindo a lista com status dos sensores, SS responde **RES_LISTSTATUS_SERVER** com essa lista, SL recebe e armazena em um buffer local para imprimir.
Se SS: envia **REQ_LISTLOC_SERVER** para SL pedindo a lista com localização dos sensores, SL responde **RES_LISTLOC_SERVER** com essa lista, SS recebe e armazena em um buffer local para imprimir.
- **locate <ID_sensor>:** Informa a localização de um sensor específico. Se for SL, já possui essa informação, se for SS, precisa solicitar ao SL.
Se SS: envia **REQ_SENSLOC** para SL, que responde com **RES_SENSLOC**.
- **status <ID_sensor>:** Informa o status de risco de um sensor específico. Se for SS, já possui essa informação, se for SL, precisa solicitar ao SS.
Se SL: envia **REQ_SENSSTATUS** para SS, que responde com **RES_SENSSTATUS**.
- **report <ID_sensor>:** Informa a localização e o status de risco de um sensor específico. Para isso, é preciso solicitar a informação faltante para o outro servidor.
Se SL: envia **REQ_RISKSSENS_SERVER** com o ID e localização para SS pedindo o status, SS responde **RES_RISKSSENS_SERVER** com ID, localização e status.
Se SS: envia **REQ_LOCSENS_SERVER** com o ID e status para SL pedindo a localização, SL responde **RES_LOCSENS_SERVER** com ID, localização e status.
- **help:** Lista comandos disponíveis para os servidores.

Comandos dos sensores:

- **close connection / kill:** Se o sensor está em alerta (status de risco 1), ele muda para status 0, já que ele vai desconectar e seu alerta não deve ser contado. Após essa verificação, solicita desconexão para os dois servidores. Se risco 1, envia **REQ_UPDRISK** com risco 0 para SS, se SS não encontrar o sensor, envia **ERROR(10)**, caso contrário SS atualiza risco do sensor e envia **REQ_LOCUPDATE** para SL, para verificar localização do sensor e atualizar a contagem de alertas da área, SL responde com **RES_LOCUPDATE** possibilitando que SS atualize a contagem de alertas da área, se área estava com pane e saiu do estado de pane após a desconexão do sensor, SS envia **REQ_MULTICAST_FAILURE** para SL informando que pane foi resolvida, SL verifica a área da troca de estado e envia **NOTIFY_FAILURE** para todos os sensores da área, informando que a pane foi resolvida. Em seguida, sensor envia **REQ_DISCSEN** para os dois servidores, que respondem **ERROR(10)** se não encontrarem o sensor, ou **OK(01)** confirmando desconexão.

- **Check failure:** Verifica se o próprio sensor está com falha.
Envia **REQ_SENSSTATUS** para SS que responde **ERROR(10)** se não encontrar o sensor. Se encontrar o sensor e o status de risco for 0, responde com **OK(03)** informando que o risco é 0. Se encontrar o sensor e o risco for 1, **REQ_CHECKALERT** para SL pedindo a localização desse sensor, SL responde com **ERROR(10)** se não encontrar sensor, caso contrário responde com **RES_CHECKALERT**, então SS responde o sensor com **RES_SENSSTATUS**, que descobre que seu status é 1, além da sua localização.
- **locate <ID_sensor>:** Informa a localização de um sensor específico. Para isso, pergunta SL.
Envia **REQ_SENSLOC** para SL, que responde **ERROR(10)** se não encontrar o sensor, caso contrário, responde com **RES_SENSLOC**.
- **diagnose <localização>:** Diagnostica uma localização verificando todos os sensores existentes nela.
Sensor envia **REQ_LOCLIST** para SL, que responde **ERROR(11)** se não encontrar a localização solicitada. Responde **ERROR(10)** se não encontrar sensor na localização. Responde **RES_LOCLIST** com lista de sensores.
- **alert / safe:** Manda alerta ou informa que não está mais em alerta.
Sensor envia **REQ_UPDRISK** com risco para SS, se SS não encontrar o sensor, envia **ERROR(10)**, caso contrário SS atualiza risco do sensor e envia **REQ_LOCUPDATE** para SL para verificar localização do sensor e atualizar a contagem de alertas da área, SL responde com **RES_LOCUPDATE** possibilitando que SS atualize a contagem de alertas da área, se área entra em estado de pane ou estava com pane e resolveu ela após a mudança de risco do sensor, SS envia **REQ_MULTICAST_FAILURE** para SL informando detecção de pane ou que ela foi resolvida, SL verifica a área da troca de estado e envia **NOTIFY_FAILURE** para todos os sensores da área, informando o novo estado da área.
- **status <ID_sensor>:** Informa o status de risco de um sensor específico. Para isso, pergunta SS.
Envia **REQ_RISKSENS_SENSOR** para SS, que responde **ERROR(10)** se não encontrar o sensor, caso contrário, responde com **RES_RISKSENS_SENSOR**.
- **list sensors:** Lista todos os sensores conectados na rede, separados por região, e informa a localização e o status de risco de cada um. Para isso, precisa consultar SL e SS.
Sensor envia **REQ_LISTLOC_SENSOR** para SL e **REQ_LISTSTATUS_SENSOR** para SS. SL responde **RES_LISTLOC_SENSOR** com a lista de localizações e SS responde **RES_LISTSTATUS_SENSOR** com a lista de status. Quando o sensor recebe a primeira resposta, ele marca qual resposta recebeu e quando recebe a segunda, ele verifica que já recebeu as duas respostas e imprime a lista completa de todos os sensores da rede.
- **report <ID_sensor>:** Informa a localização e o status de risco de um sensor específico, consultando SL e SS.
Sensor envia **REQ_REPORTLOC_SENSOR** para SL e **REQ_REPORTRISK_SENSOR** para SS. SL e SS respondem **ERROR(10)** se não encontrarem o sensor, mas se encontrarem, SL responde **RES_REPORTLOC_SENSOR** com a localização e SS responde **RES_REPORTRISK_SENSOR** com o status. Quando o sensor recebe a primeira resposta, ele marca qual resposta recebeu e quando recebe a segunda, ele verifica que já recebeu as duas respostas e imprime a localização e o status do sensor específico.
- **help:** Lista comandos disponíveis para os sensores.

Fluxos de mensagens sem comando:

- **Abertura de conexão entre servidores:** Quando um servidor é iniciado, ele tenta conectar na porta 64000. Se conseguir, estabelece conexão, caso contrário, passa a escutar por conexão nessa porta.
Se Servidor A consegue conectar na porta 64000, envia **REQ_CONNPEER** para Servidor B, então B gera um ID para A e responde com **RES_CONNPEER**, reconhece conexão e marca que está esperando por um ID também. A recebe seu ID, gera um ID para B, também responde com **RES_CONNPEER**, e reconhece conexão. Por fim, B recebe seu ID.
Se o limite de servidores foi atingido, o servidor C recebe **ERROR(01)**.
- **Abertura de conexão cliente-servidor:** Quando um sensor é iniciado, ele conecta com SL pela porta 60000 e com SS pela porta 61000.
Sensor manda **REQ_CONNSEN** para os dois servidores, os servidores respondem **ERROR(09)** se o limite de sensores foi atingido, caso contrário, enviam **RES_CONNSEN** com o ID gerado para o sensor e **OK(02)** para confirmar conexão. O sensor recebe **RES_CONNSEN** e armazena seu ID. Também recebe **OK(02)** de cada servidor, quando recebe o segundo **OK(02)**, imprime que sua criação foi um sucesso.

4. Refinamentos e Decisões de Implementação

A implementação do sistema de monitoramento de falhas exigiu o aprimoramento de certas diretrizes e a tomada de decisões específicas para garantir a robustez e o funcionamento adequado do sistema.

4.1. Abordagem Multi-threaded para Concorrência

Para gerenciar as múltiplas conexões simultâneas de clientes e a comunicação P2P entre servidores, optei por uma **abordagem multi-threaded**. Cada nova conexão de cliente recebida pelo servidor é tratada por uma thread dedicada

(handle_client). A comunicação com o servidor peer também ocorre em uma thread (handle_peer). No sensor, threads separadas (handle_server_response) são criadas para escutar as respostas de cada servidor (SL e SS). Uma thread adicional (stdin_thread) é utilizada tanto no servidor quanto no cliente para lidar com a entrada de comandos via teclado.

4.2. Geração de Identificadores (IDs)

Para garantir a unicidade dos IDs atribuídos aos servidores e sensores, foi implementada uma lógica de geração sequencial específica:

- **Servidores:** IDs no formato [tipo][porta][sequência]. Com tipo 5 para SL e 6 para SS. A porta corresponde à porta de comunicação P2P do servidor (64000), e sequência é um contador local, incrementado a cada novo ID gerado. Exemplo: 5640000001 para o primeiro SL iniciado na porta 64000.
- **Sensores:** IDs no formato [área][localização][sequência]. A área (1-4) é determinada a partir da localização do sensor, e sequência é um contador único para aquela combinação de área e localização. Se a localização for -1 (fora de zona ativa), a área é 0 e a localização no ID é 00, mas este caso não ocorre no TP. Exemplo: 103000001 para um sensor na Área Norte, localização 3, primeira sequência.

4.3. Configuração de Portas Padrão

Para padronização e organização das comunicações, foram definidas as seguintes portas:

- **Porta P2P:** A porta 64000 é utilizada para a comunicação peer-to-peer exclusiva entre SL e SS.
- **Porta SL-Sensores:** A porta 60000 é designada para o SL receber conexões e mensagens dos sensores.
- **Porta SS-Sensores:** A porta 61000 é designada para o SS receber conexões e mensagens dos sensores.

4.4. Flexibilidade da Linha de Comando do Sensor e Geração de Localização

A linha de comando do sensor é flexível na especificação das portas dos servidores e da localização inicial:

- **Portas:** O sensor pode ser iniciado passando o IP e uma única porta, ou passando o IP e ambas as portas em qualquer ordem. Caso não seja fornecida uma porta, ela é inferida com base na padronização.
 - ./sensor <ip> <porta_servidor_1>
 - ./sensor <ip> <porta_servidor_2>
 - ./sensor <ip> <porta_servidor_1> <porta_servidor_2>
 - ./sensor <ip> <porta_servidor_2> <porta_servidor_1>
- **Localização:** A localização inicial do sensor pode ser fornecida como um argumento opcional. Se não for fornecida, uma localização aleatória entre 1 e 10 é atribuída ao sensor, garantindo que ele sempre comece em uma área válida. Caso seja uma fornecida uma localização inválida, outra localização aleatória é gerada.
 - ./sensor <ip> <porta_servidor_1> <localização>
 - ./sensor <ip> <porta_servidor_2> <localização>
 - ./sensor <ip> <porta_servidor_1> <porta_servidor_2> <localização>
 - ./sensor <ip> <porta_servidor_2> <porta_servidor_1> <localização>

4.5. Criação de Comandos Extras

Para aumentar a interação com o sistema e oferecer funcionalidades de monitoramento mais abrangentes, foram implementados comandos extras para os servidores e sensores, além dos fluxos de controle e alerta básicos.

- **Comandos de Servidores:**
 - list sensors: Permite aos servidores listar todos os sensores conectados na rede, agrupados por região, informando sua localização e status de risco.
 - locate <ID_sensor>: Informa a localização de um sensor específico.
 - status <ID_sensor>: Informa o status de risco de um sensor específico.
 - report <ID_sensor>: Fornece a localização e o status de risco de um sensor específico.
 - help: Exibe uma lista de todos os comandos disponíveis para o servidor.
- **Comandos de Sensores:**
 - alert / safe: Permite ao sensor alterar seu status de risco para 1 (alerta) ou 0 (seguro).
 - status <ID_sensor>: Informa o status de risco de um sensor específico.
 - list sensors: Permite aos servidores listar todos os sensores conectados na rede, agrupados por região, informando sua localização e status de risco.
 - report <ID_sensor>: Fornece a localização e o status de risco de um sensor específico.
 - help: Exibe uma lista de todos os comandos disponíveis para o sensor.

4.6. Status Inicial de Sensores

O status inicial de risco de todos os sensores é definido como 0 (seguro). Como o status pode ser alterado com os comandos alert e safe, não há necessidade de gerar o status aleatoriamente, assim o sistema se aproxima mais da realidade.

4.7. Notificação e Detecção de Pane ou Estado Seguro da Área

A detecção de panes e sua resolução são funcionalidades cruciais, que cumprem com o objetivo do trabalho. Quando mais de 2 sensores de uma área enviam alerta, SS detecta pane e notifica SL que, por sua vez, notifica todos os sensores da área. Quando os sensores que estão em alerta em uma área de pane indicam segurança, de forma que restam apenas 2 ou menos sensores em alerta, SS considera que a pane foi resolvida e informa SL que, por sua vez, informa todos os sensores da área. Observe um exemplo no fim do arquivo, na seção 6. Imagens.

4.8. Gerenciamento de Desconexões

A lógica de desconexão foi aprimorada para garantir a resiliência do sistema:

- **Desconexão de Servidores:** Quando um comando kill ou close connection é emitido para um servidor, ele notifica automaticamente todos os sensores conectados. Isso permite que os sensores se encerrem de forma controlada, reconhecendo que a infraestrutura de monitoramento está sendo desativada.
- **Reconexão de Servidor Peer:** Em caso de desconexão do peer, o servidor tenta reiniciar a lógica de conexão (start_server) para restabelecer a comunicação, seja conectando-se a um novo peer ou voltando a escutar por novas conexões.
- **Tratamento de Risco ao Desconectar Sensor:** Se um sensor estiver em estado de alerta (risk == 1) e receber um comando de desconexão (kill / close connection), ele primeiro altera seu risco para 0. Isso garante que o alerta desse sensor seja "desarmado" no sistema antes de sua saída, evitando que ele continue na contagem de panes

5. Conclusão

O sistema de monitoramento de falhas em redes elétricas foi desenvolvido para simular um ambiente de detecção de anomalias, com comunicação eficiente entre sensores (clientes) e servidores de localização (SL) e de status (SS), incluindo interação P2P entre os próprios servidores. A implementação em C, utilizando sockets POSIX e multi-threading, garantiu o funcionamento para o gerenciamento de múltiplas conexões e fluxos de mensagens.

Para compilar e executar o sistema:

- Acesse o diretório TP_2020007112.
- Compile o projeto no terminal com o comando: make. Isso gerará os executáveis server.exe e sensor.exe.
- Utilize terminais distintos para iniciar os servidores e sensores:
- Servidor SL: Terminal 1: ./server 127.0.0.1 64000 60000.
- Servidor SS: Terminal 2: ./server 127.0.0.1 64000 61000.
- Sensores (até 15): Em Terminais N, use ./sensor <ip> <porta_servidor_1> [porta_servidor_2] [LocId]. (Seção 4.4)

6. Imagens

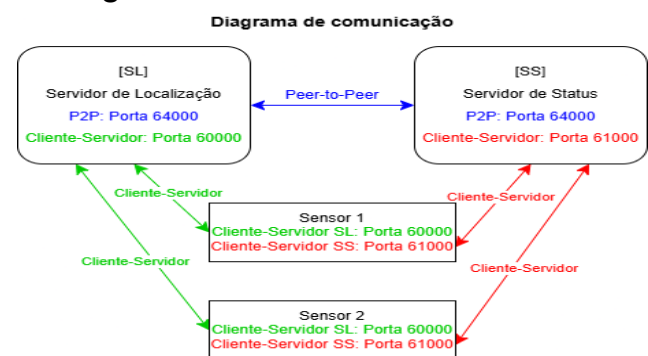


Imagem 1: Diagrama de comunicação. Mostra como são realizadas as conexões Peer-to-Peer e Cliente-Servidor.

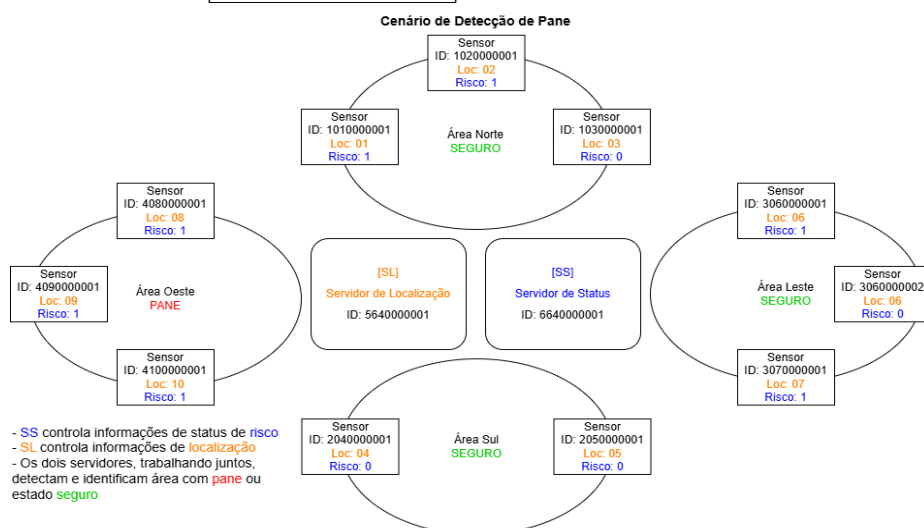


Imagem 2: Cenário de Detecção de pane. Exemplifica uma rede de monitoramento, onde foi detectado pane na área Oeste.